Experiment 08 — High-Performance Big Data Analytics Using R

Name: Sheshank Singh
Roll Number: 23102A0017
Subject: R Programming

Problem Statement

The New York City Taxi and Limousine Commission (NYC TLC) generates millions of taxi-trip records containing pickup and drop-off timestamps, locations, passenger counts, trip distance, fare, payment type and total amount. Processing such large datasets with conventional approaches can increase execution time and memory consumption.

The objective of this experiment is to build a scalable analytics workflow in R using the NYC Yellow Taxi Trip dataset. The experiment uses `data.table` for efficient data processing, compares functional and vectorized implementations, evaluates sequential and parallel processing, and benchmarks alternative approaches.

Objectives

- Import, clean and prepare NYC Yellow Taxi Trip data.
- Join trip records with the official Taxi Zone Lookup Table.
- Analyse demand, revenue, routes, fare, distance and payment preferences.
- Compare base R, functional programming, vectorized operations and `data.table`.
- Compare sequential and parallel execution using `foreach` and `doParallel`.
- Measure execution time and calculate parallel speedup.
- Visualize important findings using `ggplot2`.
- Recommend suitable approaches based on measured results.

Dataset Description

Dataset: NYC Taxi & Limousine Commission (TLC) — Yellow Taxi Trip Records
Official source: NYC TLC Trip Record Data

This notebook downloads the January 2024 Yellow Taxi Trip Records in Parquet format and the official Taxi Zone Lookup Table.

1. Install and Load Required Packages

The packages below support Parquet data import, efficient data manipulation, visualization, functional programming, benchmarking and parallel processing.

```
packages <- c(
  "arrow",
  "data.table",
  "ggplot2",
  "foreach",
  "doParallel",
  "microbenchmark",
  "purrr",
  "lubridate"
)

installed <- rownames(installed.packages())
for (pkg in packages) {
  if (!pkg %in% installed) {
    install.packages(pkg, repos = "https://cloud.r-project.org")
  }
}

library(arrow)
library(data.table)
library(ggplot2)
library(foreach)
library(doParallel)
library(microbenchmark)
library(purrr)
library(lubridate)

cat("All required packages are loaded.\n")
```

```
All required packages are loaded.
```

2. Download the Taxi Dataset and Zone Lookup

The official TLC monthly Parquet file and zone lookup table are downloaded directly. If the files already exist in the session, they are reused.

```
trip_url <- "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata
zone_url <- "https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv

trip_file <- "yellow_tripdata_2024-01.parquet"
zone_file <- "taxi_zone_lookup.csv"

if (!file.exists(trip_file)) {
  download.file(trip_url, trip_file, mode = "wb", quiet = FALSE)
}
```

```
if (!file.exists(zone_file)) {
  download.file(zone_url, zone_file, mode = "wb", quiet = FALSE)
}

cat("Trip file exists:", file.exists(trip_file), "\n")
cat("Zone lookup exists:", file.exists(zone_file), "\n")
```

```
Trip file exists: TRUE
Zone lookup exists: TRUE
```

3. Import and Inspect the Data

The Parquet file is read into R. A configurable sample limit helps control memory use in a hosted notebook environment.

```
MAX_ROWS <- 500000  # Change to Inf to retain the full monthly file if memor

trips_raw <- as.data.table(arrow::read_parquet(trip_file))
zones <- fread(zone_file)

cat("Rows in downloaded monthly file:", format(nrow(trips_raw), big.mark = "
cat("Columns:", ncol(trips_raw), "\n")

if (is.finite(MAX_ROWS) && nrow(trips_raw) > MAX_ROWS) {
  set.seed(123)
  trips <- trips_raw[sample.int(.N, MAX_ROWS)]
} else {
  trips <- copy(trips_raw)
}

cat("Rows selected for this experiment:", format(nrow(trips), big.mark = ","
cat("Zone lookup rows:", nrow(zones), "\n")
```

```
Rows in downloaded monthly file: 2,964,624
Columns: 19
Rows selected for this experiment: 500,000
Zone lookup rows: 265
```

```
# Inspect the structure and data types
str(trips)
```

```
Classes ‘data.table’ and 'data.frame':  500000 obs. of  19 variables:
 $ VendorID             : int  2 2 2 2 2 2 2 2 1 2 ...
 $ tpep_pickup_datetime : POSIXct, format: "2024-01-23 18:08:56" "2024-01-02
 $ tpep_dropoff_datetime: POSIXct, format: "2024-01-23 18:15:46" "2024-01-02
 $ passenger_count      : int  2 1 1 1 1 2 1 2 3 1 ...
 $ trip_distance        : num  0.53 13.11 1.76 1.72 1.61 ...
 $ RatecodeID           : int  1 1 1 1 1 1 1 3 1 1 ...
 $ store_and_fwd_flag   : chr  "N" "N" "N" "N" ...
 $ PULocationID         : int  237 132 186 114 166 48 238 161 4 142 ...
 $ DOLocationID         : int  237 252 229 234 43 163 90 1 264 142 ...
 $ payment_type         : int  1 2 1 1 1 1 1 2 1 1 ...
 $ fare_amount          : num  7.9 49.9 17.7 12.8 10.7 17.7 25.4 88.8 7.2 5.1
```

```
 $ extra                : num  2.5 1 2.5 1 0 0 2.5 1 3.5 0 ...
 $ mta_tax              : num  0.5 0.5 0.5 0.5 0.5 0.5 0.5 0 0.5 0.5 ...
 $ tip_amount           : num  3.6 0 7.26 3.56 2.44 2 4 0 3 1.82 ...
 $ tolls_amount         : num  0 0 0 0 0 ...
 $ improvement_surcharge: num  1 1 1 1 1 1 1 1 1 1 ...
 $ total_amount         : num  18 54.1 31.5 21.4 14.6 ...
 $ congestion_surcharge : num  2.5 0 2.5 2.5 0 2.5 2.5 0 2.5 2.5 ...
 $ Airport_fee          : num  0 1.75 0 0 0 0 0 0 0 0 ...
 - attr(*, ".internal.selfref")=<externalptr>
```

```
# Dataset dimensions and summary statistics
dim(trips)
summary(trips)
```

```
500000 · 19
    VendorID     tpep_pickup_datetime
 Min.   :1.000   Min.   :2002-12-31 22:59:39.00
 1st Qu.:2.000   1st Qu.:2024-01-09 15:37:53.50
 Median :2.000   Median :2024-01-17 10:55:11.00
 Mean   :1.753   Mean   :2024-01-17 00:38:27.57
 3rd Qu.:2.000   3rd Qu.:2024-01-24 18:38:53.75
 Max.   :6.000   Max.   :2024-01-31 23:59:30.00

 tpep_dropoff_datetime            passenger_count trip_distance
 Min.   :2002-12-31 23:05:41.00   Min.   :0.000   Min.   :    0.00
 1st Qu.:2024-01-09 15:53:48.00   1st Qu.:1.000   1st Qu.:    1.00
 Median :2024-01-17 11:13:40.50   Median :1.000   Median :    1.68
 Mean   :2024-01-17 00:54:00.96   Mean   :1.338   Mean   :    3.34
 3rd Qu.:2024-01-24 18:54:33.00   3rd Qu.:1.000   3rd Qu.:    3.13
 Max.   :2024-02-01 22:04:34.00   Max.   :8.000   Max.   :33916.10
                                  NA's   :23679
   RatecodeID     store_and_fwd_flag  PULocationID  DOLocationID
 Min.   : 1.000   Length:500000      Min.   :  1   Min.   :  1.0
 1st Qu.: 1.000   Class :character   1st Qu.:132   1st Qu.:114.0
 Median : 1.000   Mode  :character   Median :162   Median :162.0
 Mean   : 2.069                      Mean   :166   Mean   :164.9
 3rd Qu.: 1.000                      3rd Qu.:234   3rd Qu.:234.0
 Max.   :99.000                      Max.   :265   Max.   :265.0
 NA's   :23679
  payment_type    fare_amount          extra           mta_tax
 Min.   :0.000   Min.   :-800.00   Min.   :-7.500   Min.   :-0.5000
 1st Qu.:1.000   1st Qu.:   8.60   1st Qu.: 0.000   1st Qu.: 0.5000
 Median :1.000   Median :  12.80   Median : 1.000   Median : 0.5000
 Mean   :1.161   Mean   :  18.22   Mean   : 1.456   Mean   : 0.4834
 3rd Qu.:1.000   3rd Qu.:  20.50   3rd Qu.: 2.500   3rd Qu.: 0.5000
 Max.   :4.000   Max.   :2500.00   Max.   :11.750   Max.   : 4.0000

   tip_amount      tolls_amount      improvement_surcharge  total_amount
 Min.   :-65.10   Min.   :-50.0000   Min.   :-1.0000       Min.   :-801.00
 1st Qu.:  1.00   1st Qu.:  0.0000   1st Qu.: 1.0000       1st Qu.:  15.38
 Median :  2.70   Median :  0.0000   Median : 1.0000       Median :  20.10
 Mean   :  3.34   Mean   :  0.5312   Mean   : 0.9758       Mean   :  26.86
 3rd Qu.:  4.12   3rd Qu.:  0.0000   3rd Qu.: 1.0000       3rd Qu.:  28.56
 Max.   :150.00   Max.   : 85.0000   Max.   : 1.0000       Max.   :2500.00

 congestion_surcharge  Airport_fee
 Min.   :-2.500       Min.   :-1.750
 1st Qu.: 2.500       1st Qu.: 0.000
 Median : 2.500       Median : 0.000
 Mean   : 2.255       Mean   : 0.143
 3rd Qu.: 2.500       3rd Qu.: 0.000
 Max.   : 2.500       Max.   : 1.750
 NA's   :23679        NA's   :23679
```

```
# Check missing values by column
missing_summary <- data.table(
  Column = names(trips),
  Missing_Count = vapply(trips, function(x) sum(is.na(x)), numeric(1))
)
missing_summary[order(-Missing_Count)]
```

A data.table: 19 × 2

Column	Missing_Count
<chr>	<dbl>
passenger_count	23679
RatecodeID	23679
store_and_fwd_flag	23679
congestion_surcharge	23679
Airport_fee	23679
VendorID	0
tpep_pickup_datetime	0
tpep_dropoff_datetime	0
trip_distance	0
PULocationID	0
DOLocationID	0
payment_type	0
fare_amount	0
extra	0
mta_tax	0
tip_amount	0
tolls_amount	0
improvement_surcharge	0
total_amount	0

4. Data Cleaning and Feature Engineering

The cleaning process removes exact duplicate rows and records with invalid timestamps, negative/zero trip distance, invalid passenger counts, and non-positive total amounts. Some zero-passenger records may be valid in real data, so this notebook retains passenger counts from 0 to 6 and reports the cleaning effect. Temporal fields are extracted for hourly, weekday and monthly analysis.

```
# Standardize the data.table object and select useful columns
setDT(trips)

required_cols <- c(
  "VendorID", "tpep_pickup_datetime", "tpep_dropoff_datetime",
  "passenger_count", "trip_distance", "RatecodeID",
```

```
  "PULocationID", "DOLocationID", "payment_type",
  "fare_amount", "tip_amount", "tolls_amount", "total_amount"
)
missing_cols <- setdiff(required_cols, names(trips))
if (length(missing_cols) > 0) {
  stop(paste("Expected columns missing:", paste(missing_cols, collapse = ",
}
trips <- trips[, ..required_cols]

rows_before <- nrow(trips)
trips <- unique(trips)

trips[, tpep_pickup_datetime := as.POSIXct(tpep_pickup_datetime, tz = "UTC")
trips[, tpep_dropoff_datetime := as.POSIXct(tpep_dropoff_datetime, tz = "UTC

trips_clean <- trips[
  !is.na(tpep_pickup_datetime) &
  !is.na(tpep_dropoff_datetime) &
  tpep_dropoff_datetime >= tpep_pickup_datetime &
  !is.na(PULocationID) & !is.na(DOLocationID) &
  !is.na(trip_distance) & trip_distance > 0 &
  !is.na(fare_amount) & fare_amount > 0 &
  !is.na(total_amount) & total_amount > 0 &
  passenger_count >= 0 & passenger_count <= 6
]

cat("Rows before cleaning:", format(rows_before, big.mark = ","), "\n")
cat("Rows after cleaning:", format(nrow(trips_clean), big.mark = ","), "\n")
cat("Rows removed:", format(rows_before - nrow(trips_clean), big.mark = ",")
```

```
Rows before cleaning: 500,000
Rows after cleaning: 464,531
Rows removed: 35,469
```

```
# Extract time-based features and calculate trip duration
trips_clean[, `:=`(
  pickup_hour = hour(tpep_pickup_datetime),
  pickup_day = day(tpep_pickup_datetime),
  pickup_weekday = wday(tpep_pickup_datetime, label = TRUE, abbr = FALSE, we
  pickup_month = month(tpep_pickup_datetime, label = TRUE, abbr = TRUE),
  trip_duration_min = as.numeric(difftime(
    tpep_dropoff_datetime, tpep_pickup_datetime, units = "mins"
  ))
)]

# Remove implausible duration values for analysis
trips_clean <- trips_clean[
  trip_duration_min >= 0 & trip_duration_min <= 360
]

# Retain only columns required for the rest of the analysis
trips_clean <- trips_clean[, .(
  tpep_pickup_datetime, tpep_dropoff_datetime,
  passenger_count, trip_distance, PULocationID, DOLocationID,
```

```
  payment_type, fare_amount, tip_amount, tolls_amount, total_amount,
  pickup_hour, pickup_day, pickup_weekday, pickup_month, trip_duration_min
)]

cat("Cleaned records available for analysis:", format(nrow(trips_clean), big
head(trips_clean)
```

```
Cleaned records available for analysis: 464,266
```

tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance
<dttm>	<dttm>	<int>	<dbl>
2024-01-23 18:08:56	2024-01-23 18:15:46	2	0.53
2024-01-02 20:25:01	2024-01-02 20:49:11	1	13.11
2024-01-31 16:57:30	2024-01-31 17:18:11	1	1.76
2024-01-20 02:29:24	2024-01-20 02:40:23	1	1.72
2024-01-22 09:15:49	2024-01-22 09:24:15	1	1.61
2024-01-06 19:23:01	2024-01-06 19:41:48	2	1.56

5. Integrate Taxi Zone Information

The Taxi Zone Lookup Table maps location IDs to zone names and boroughs. Two copies are joined to the trip data: one for pickup and one for drop-off locations.

```
zones_pickup <- copy(zones)
setnames(
  zones_pickup,
  old = c("LocationID", "Borough", "Zone"),
  new = c("PULocationID", "PU_Borough", "PU_Zone")
)

zones_dropoff <- copy(zones)
setnames(
  zones_dropoff,
  old = c("LocationID", "Borough", "Zone"),
  new = c("DOLocationID", "DO_Borough", "DO_Zone")
)

trips_joined <- merge(trips_clean, zones_pickup, by = "PULocationID", all.x
trips_joined <- merge(trips_joined, zones_dropoff, by = "DOLocationID", all.

cat("Rows after zone joins:", format(nrow(trips_joined), big.mark = ","), "\
cat("Missing pickup zone names:", sum(is.na(trips_joined$PU_Zone)), "\n")
cat("Missing drop-off zone names:", sum(is.na(trips_joined$DO_Zone)), "\n")
head(trips_joined)
```

```
Rows after zone joins: 464,266
Missing pickup zone names: 0
Missing drop-off zone names: 0
```

DOLocationID	PULocationID	tpep_pickup_datetime	tpep_dropoff_datetime	pass
<int>	<int>	<dttm>	<dttm>	
1	1	2024-01-01 17:45:45	2024-01-01 17:50:13	
1	1	2024-01-25 13:07:20	2024-01-25 13:07:32	
1	1	2024-01-21 08:46:54	2024-01-21 08:47:22	
1	1	2024-01-18 18:48:04	2024-01-18 18:48:19	
1	1	2024-01-03 08:00:05	2024-01-03 08:00:10	
1	1	2024-01-01 14:57:56	2024-01-01 14:58:47	

6. Transportation Data Analytics

The following summaries examine taxi demand by hour and weekday, monthly usage, average fares, revenue, common routes, high-revenue routes and payment preferences. The results are calculated from the selected and cleaned records, not from the entire TLC archive.

```
# Number of trips by hour
trips_by_hour <- trips_joined[, .(
  Trips = .N,
  Average_Fare = mean(fare_amount, na.rm = TRUE),
  Total_Revenue = sum(total_amount, na.rm = TRUE)
), by = pickup_hour][order(pickup_hour)]

trips_by_hour
```

A data.table: 24 × 4

pickup_hour	Trips	Average_Fare	Total_Revenue
<int>	<int>	<dbl>	<dbl>
0	11860	19.63224	341136.50
1	7725	17.26633	199383.06
2	5572	16.57241	138453.77
3	3532	17.77266	92615.59
4	2100	23.32846	69223.02
5	2666	28.22398	103025.86
6	6128	21.81190	183575.30
7	12793	18.57849	339012.25
8	18060	17.70290	461763.16
9	20439	17.81005	527460.14
10	22090	18.05607	575588.55
11	23789	17.78648	611469.53
12	26028	17.81428	671840.87
13	27092	18.39244	718742.35
14	29289	19.29322	812359.91
15	30365	18.78574	820455.90
16	30181	19.46825	901348.78
17	32468	18.00749	913910.02
18	33155	16.94037	892311.82
19	29295	17.86459	823795.90
20	25457	18.10185	698558.52
21	25302	18.50115	704316.41
22	22464	19.20797	645995.04
23	16416	20.37855	492484.01

```
# Demand by weekday
trips_by_weekday <- trips_joined[, .(
  Trips = .N,
  Total_Revenue = sum(total_amount, na.rm = TRUE),
  Average_Fare = mean(fare_amount, na.rm = TRUE)
), by = pickup_weekday][order(pickup_weekday)]

trips_by_weekday
```

A data.table: 7 × 4

pickup_weekday	Trips	Total_Revenue	Average_Fare
<ord>	<int>	<dbl>	<dbl>
Monday	63253	1838917	19.75405
Tuesday	72771	2012034	18.50261
Wednesday	78318	2160371	18.42287
Thursday	67564	1877634	18.57762
Friday	64672	1763617	18.18541
Saturday	65523	1651096	17.13303
Sunday	52165	1435157	18.77916

```
# Monthly usage (selected file may contain only one month)
trips_by_month <- trips_joined[, .(
  Trips = .N,
  Total_Revenue = sum(total_amount, na.rm = TRUE),
  Average_Fare = mean(fare_amount, na.rm = TRUE)
), by = pickup_month]

trips_by_month
```

A data.table: 2 × 4

pickup_month	Trips	Total_Revenue	Average_Fare
<ord>	<int>	<dbl>	<dbl>
Jan	464264	12738795.31	18.46421
Dec	2	30.95	8.60000

```
# Most frequently travelled pickup-drop-off routes
top_routes <- trips_joined[, .(
  Trips = .N,
  Total_Revenue = sum(total_amount, na.rm = TRUE),
  Average_Fare = mean(fare_amount, na.rm = TRUE)
), by = .(PU_Zone, DO_Zone)][order(-Trips)][1:min(10, .N)]

top_routes
```

A data.table: 10 × 5

PU_Zone	DO_Zone	Trips	Total_Revenue	Average_Fare
<chr>	<chr>	<int>	<dbl>	<dbl>
Upper East Side South	Upper East Side North	3458	54086.32	8.770648
Upper East Side North	Upper East Side South	3154	50695.77	9.290457
Upper East Side North	Upper East Side North	2527	32949.15	6.681717
Upper East Side South	Upper East Side South	2353	32605.99	7.370484
Midtown Center	Upper East Side South	1681	28731.29	10.000833
Lincoln Square East	Upper West Side South	1499	22314.82	8.002735
Midtown Center	Upper East Side North	1484	32601.11	13.975000
Upper East Side South	Midtown Center	1473	24164.43	9.666395
Upper West Side South	Upper West Side North	1426	19862.65	7.193478

```
# Highest-revenue pickup-drop-off routes
top_revenue_routes <- trips_joined[, .(
  Trips = .N,
  Total_Revenue = sum(total_amount, na.rm = TRUE),
  Average_Fare = mean(fare_amount, na.rm = TRUE)
), by = .(PU_Zone, DO_Zone)][order(-Total_Revenue)][1:min(10, .N)]

top_revenue_routes
```

A data.table: 10 × 5

PU_Zone	DO_Zone	Trips	Total_Revenue	Average_Fare
<chr>	<chr>	<int>	<dbl>	<dbl>
JFK Airport	Outside of NYC	975	124011.99	107.120021
JFK Airport	Times Sq/Theatre District	986	92714.56	70.372485
LaGuardia Airport	Times Sq/Theatre District	929	70886.96	47.187901
Upper East Side South	Upper East Side North	3458	54086.32	8.770648
JFK Airport	Clinton East	562	52708.54	70.473434
Upper East Side North	Upper East Side South	3154	50695.77	9.290457
JFK Airport	Midtown South	531	50560.40	70.108832
Times Sq/Theatre District	LaGuardia Airport	593	44978.92	49.461551
Times Sq/Theatre				

```
# Payment preferences by borough
payment_by_borough <- trips_joined[, .(
  Trips = .N,
  Revenue = sum(total_amount, na.rm = TRUE)
), by = .(PU_Borough, payment_type)][order(PU_Borough, -Trips)]

payment_by_borough
```

A data.table: 27 × 4

PU_Borough	payment_type	Trips	Revenue
<chr>	<int>	<int>	<dbl>
Bronx	1	762	28920.02
Bronx	2	69	1723.12
Bronx	4	7	241.94
Bronx	3	5	89.04
Brooklyn	1	2460	83695.06
Brooklyn	2	258	6769.08
Brooklyn	4	29	568.58
Brooklyn	3	8	158.10
EWR	1	10	1210.63
Manhattan	1	348850	8113440.52
Manhattan	2	62298	1208324.67
Manhattan	4	3162	61583.01
Manhattan	3	1488	26254.18
N/A	1	62	7136.21
N/A	2	17	1051.75
N/A	4	5	401.21
N/A	3	5	161.61
Queens	1	33836	2627598.77
Queens	2	8514	483277.10
Queens	4	533	29348.16
Queens	3	250	10996.63
Staten Island	2	2	40.16
Staten Island	1	1	103.94
Unknown	1	1310	38411.97
Unknown	2	307	6921.60
Unknown	3	12	250.50
Unknown	4	6	148.70

```
# Overall fare and trip-distance summary
fare_distance_summary <- trips_joined[, .(
  Trips = .N,
  Mean_Distance = mean(trip_distance, na.rm = TRUE),
```

```
  Median_Distance = median(trip_distance, na.rm = TRUE),
  Mean_Fare = mean(fare_amount, na.rm = TRUE),
  Median_Fare = median(fare_amount, na.rm = TRUE),
  Total_Revenue = sum(total_amount, na.rm = TRUE)
)]

fare_distance_summary
```

A data.table: 1 × 6

Trips	Mean_Distance	Median_Distance	Mean_Fare	Median_Fare	Total_Revenue
<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
464266	3.302791	1.7	18.46417	12.8	12738826

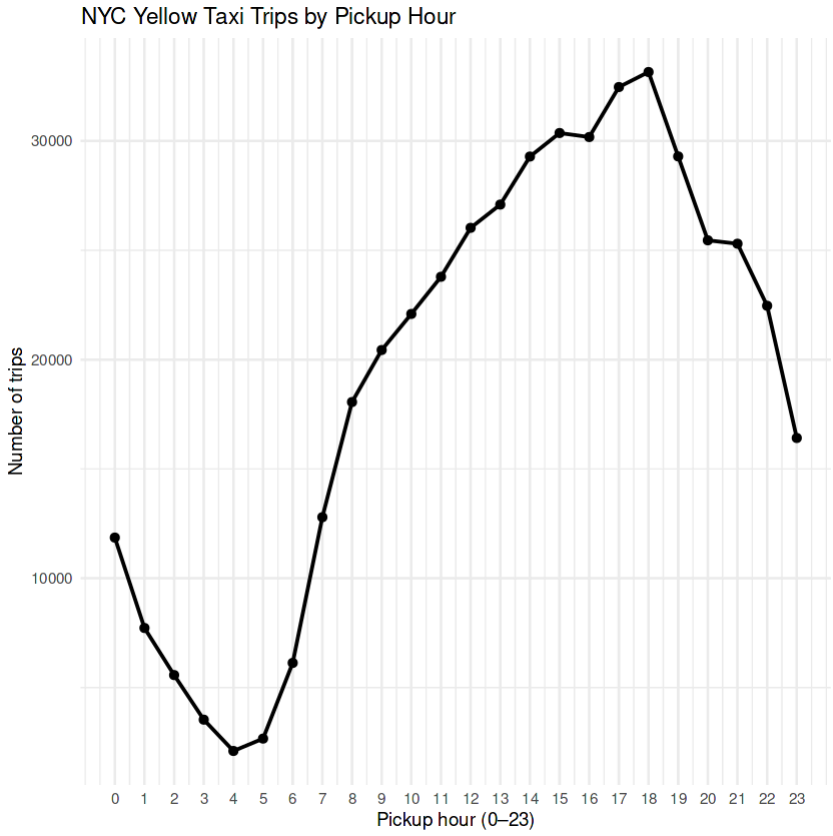
Interpretation of Transportation Analytics

Review the tables above after running the notebook. The most frequent route and highest-revenue route are not necessarily the same: one measures trip volume, while the other measures collected revenue. Hourly and weekday summaries reveal demand variation in the selected data. Payment comparisons should be interpreted alongside any missing or unknown borough/location values.

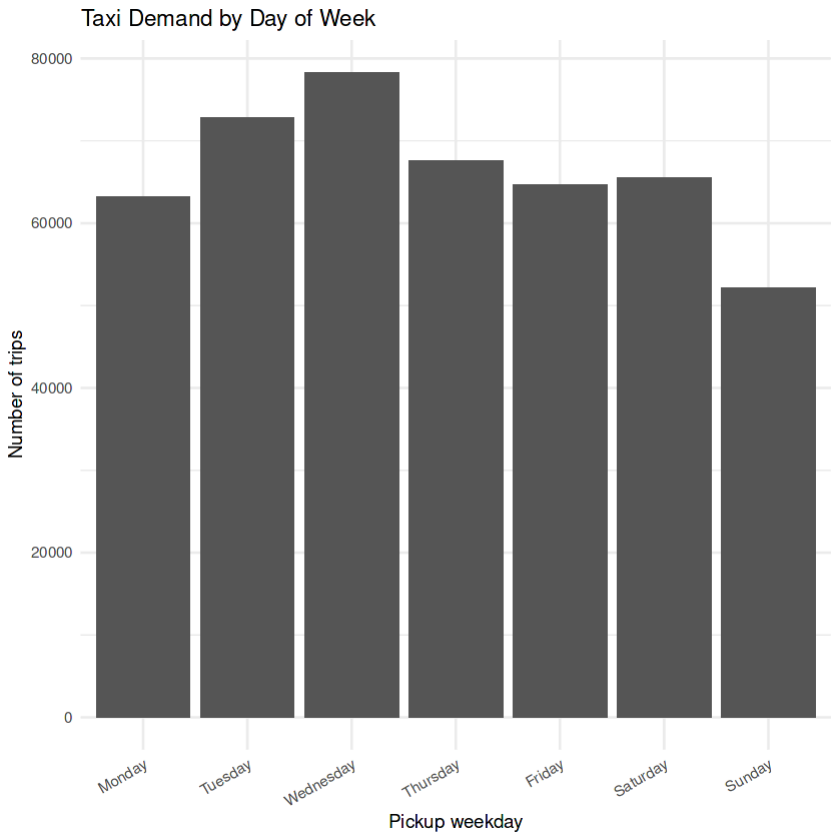
7. Visualizations Using ggplot2

Each chart includes a title, axis labels and units. These charts are generated from the computed summaries.

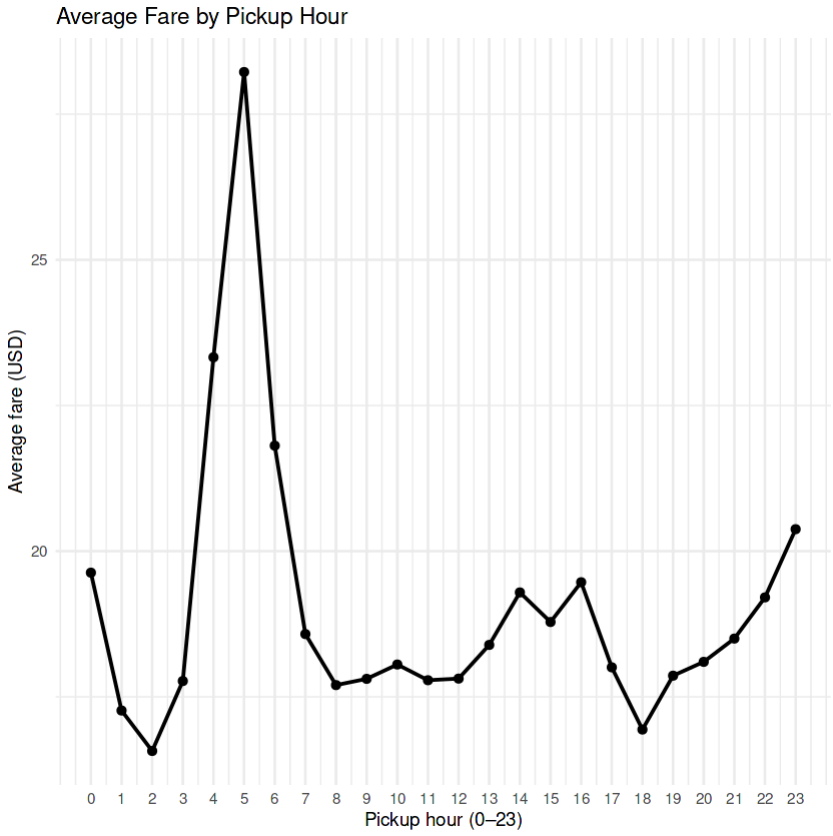
```
ggplot(trips_by_hour, aes(x = pickup_hour, y = Trips)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.8) +
  scale_x_continuous(breaks = 0:23) +
  labs(
    title = "NYC Yellow Taxi Trips by Pickup Hour",
    x = "Pickup hour (0–23)",
    y = "Number of trips"
  ) +
  theme_minimal()
```



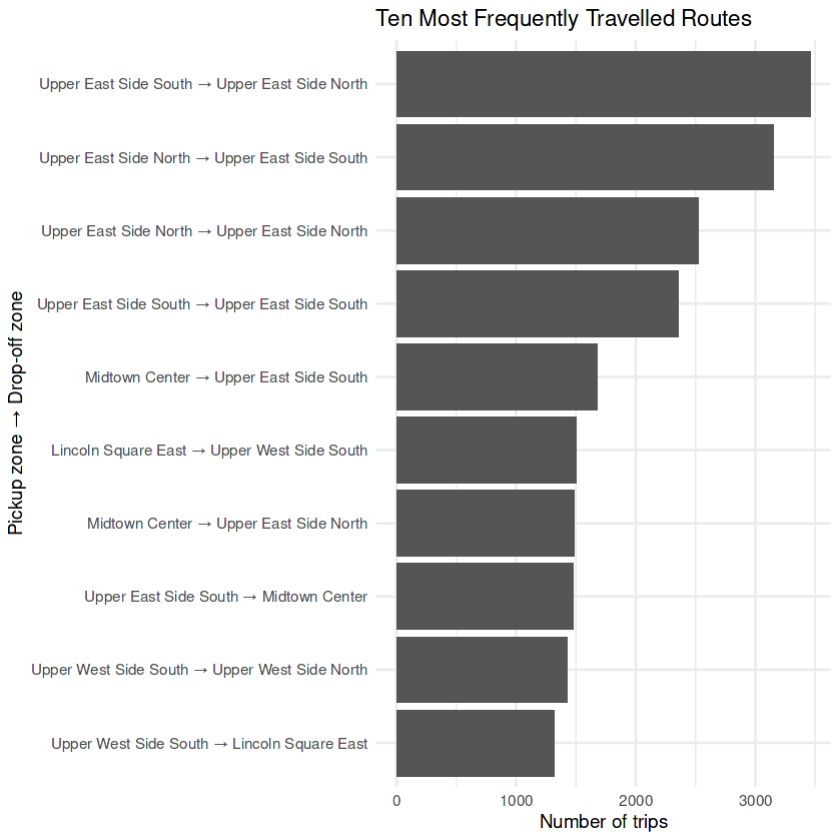

```
ggplot(trips_by_weekday, aes(x = pickup_weekday, y = Trips)) +
  geom_col() +
  labs(
    title = "Taxi Demand by Day of Week",
    x = "Pickup weekday",
    y = "Number of trips"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
```



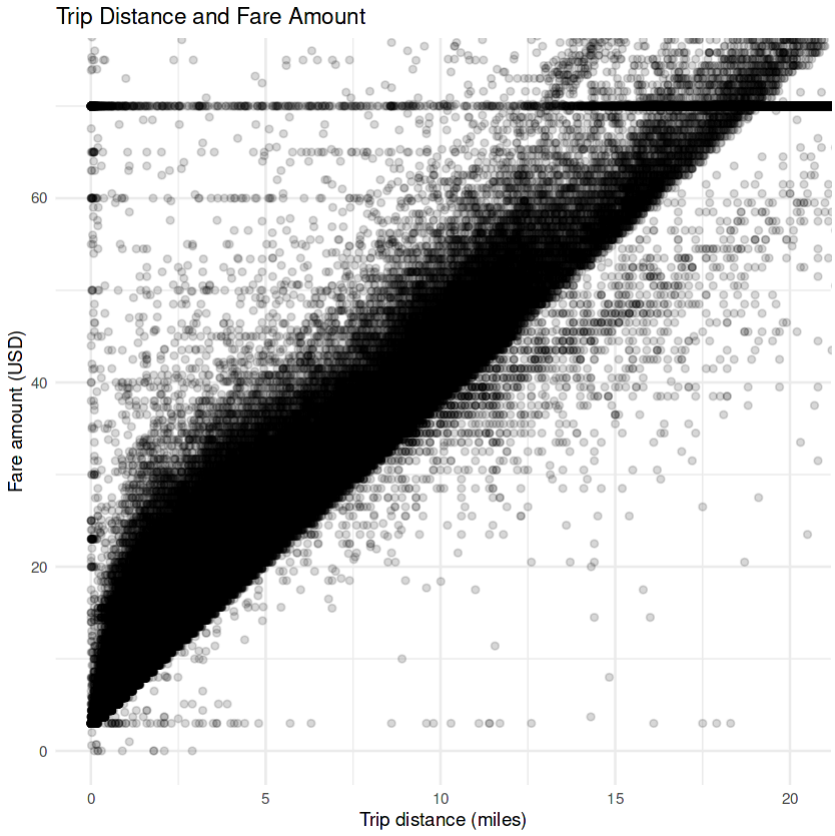

```
ggplot(trips_by_hour, aes(x = pickup_hour, y = Average_Fare)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.8) +
  scale_x_continuous(breaks = 0:23) +
  labs(
    title = "Average Fare by Pickup Hour",
    x = "Pickup hour (0–23)",
    y = "Average fare (USD)"
  ) +
  theme_minimal()
```



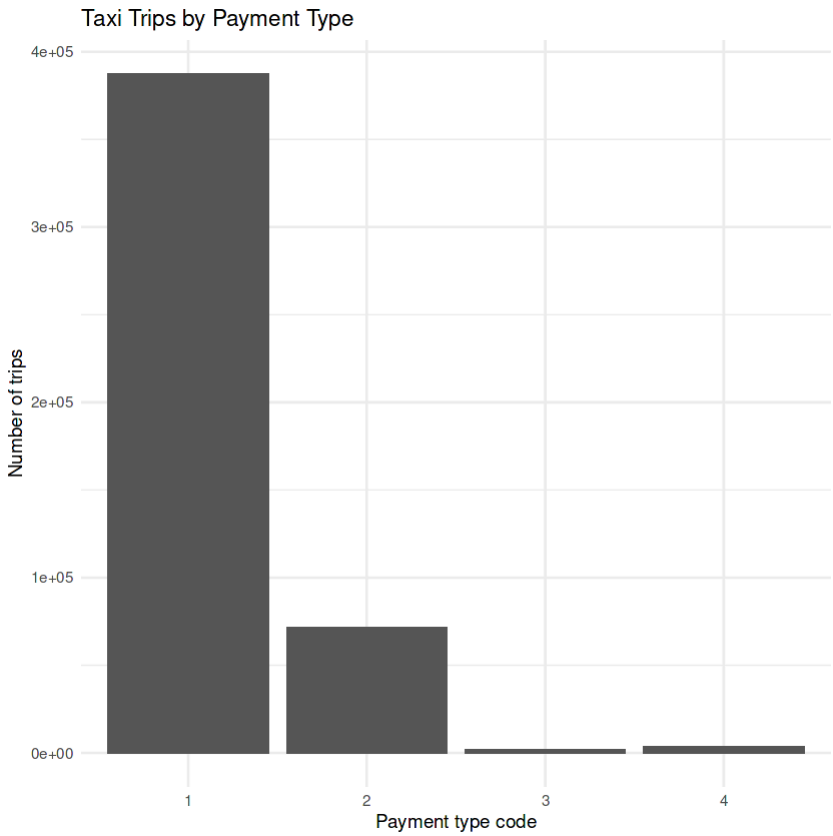

```
ggplot(top_routes, aes(x = reorder(paste(PU_Zone, "→", DO_Zone), Trips), y =
  geom_col() +
  coord_flip() +
  labs(
    title = "Ten Most Frequently Travelled Routes",
    x = "Pickup zone → Drop-off zone",
    y = "Number of trips"
  ) +
  theme_minimal()
```




```
ggplot(trips_joined, aes(x = trip_distance, y = fare_amount)) +
  geom_point(alpha = 0.15) +
  coord_cartesian(xlim = c(0, quantile(trips_joined$trip_distance, 0.99, na.
                  ylim = c(0, quantile(trips_joined$fare_amount, 0.99, na.rm
  labs(
    title = "Trip Distance and Fare Amount",
    x = "Trip distance (miles)",
    y = "Fare amount (USD)"
  ) +
  theme_minimal()
```




```
payment_summary <- trips_joined[, .(Trips = .N), by = payment_type]
ggplot(payment_summary, aes(x = factor(payment_type), y = Trips)) +
  geom_col() +
  labs(
    title = "Taxi Trips by Payment Type",
    x = "Payment type code",
    y = "Number of trips"
  ) +
  theme_minimal()
```



8. Functional Programming, Vectorization and data.table

Equivalent operations are implemented using base R, `lapply()` / `purrr::map()`, vectorized operations and `data.table`. The examples calculate trip counts by pickup hour. Benchmarking uses the same subset and produces actual timing results in the current runtime.

```
# Use a moderate benchmark sample so comparisons finish promptly
set.seed(123)
BENCH_ROWS <- min(100000L, nrow(trips_joined))
bench_data <- as.data.frame(trips_joined[sample.int(nrow(trips_joined), BENC

# Base R table implementation
base_hour_counts <- function(d) {
  as.data.frame(table(d$pickup_hour), stringsAsFactors = FALSE)
}

# Vectorized tabulation implementation
vectorized_hour_counts <- function(d) {
  counts <- tabulate(d$pickup_hour + 1L, nbins = 24L)
  data.frame(pickup_hour = 0:23, Trips = counts)
}

# data.table grouping implementation
datatable_hour_counts <- function(d) {
  as.data.table(d)[, .(Trips = .N), by = pickup_hour][order(pickup_hour)]
}
```

```
# Functional approach: lapply over hour partitions
lapply_hour_counts <- function(d) {
  hour_groups <- split(d, d$pickup_hour)
  counts <- lapply(hour_groups, nrow)
  data.frame(
    pickup_hour = as.integer(names(counts)),
    Trips = as.integer(unlist(counts)),
    row.names = NULL
  )
}

# purrr::map over hour partitions
purrr_hour_counts <- function(d) {
  hour_groups <- split(d, d$pickup_hour)
  counts <- purrr::map_int(hour_groups, nrow)
  data.frame(
    pickup_hour = as.integer(names(counts)),
    Trips = as.integer(counts),
    row.names = NULL
  )
}

# Validate key totals before benchmarking
cat("Benchmark rows:", nrow(bench_data), "\n")
cat("Vectorized count total:", sum(vectorized_hour_counts(bench_data)$Trips)
cat("data.table count total:", sum(datatable_hour_counts(bench_data)$Trips),
```

```
Benchmark rows: 100000
Vectorized count total: 100000
data.table count total: 100000
```

```
# Compare execution times using microbenchmark
benchmark_results <- microbenchmark(
  Base_R = base_hour_counts(bench_data),
  Vectorized = vectorized_hour_counts(bench_data),
  lapply = lapply_hour_counts(bench_data),
  purrr_map = purrr_hour_counts(bench_data),
  data_table = datatable_hour_counts(bench_data),
  times = 5L
)

benchmark_results
```

A microbenchmark: 25 × 2

expr	time
<fct>	<dbl>
data_table	10622567
Vectorized	4827355
Base_R	9487206
Base_R	11219938
lapply	48040080
lapply	53737774
purrr_map	47024155
data_table	6504719
Vectorized	973748
Vectorized	866488
purrr_map	53758121
Vectorized	938916
data_table	6497354
Vectorized	948356
data_table	6635014
Base_R	8481641
purrr_map	48804228
Base_R	8453882
lapply	50118261
data_table	6815307
lapply	49364961
Base_R	8564091
purrr_map	51159361
lapply	49456518
purrr_map	51218785

```
# Summarize benchmark timings in milliseconds
benchmark_summary <- as.data.frame(summary(benchmark_results))
benchmark_summary[, c("expr", "min", "median", "mean", "max")]
```

A data.frame: 5 × 5